Post-quantum verifiable credentials: SNARK vs zkVM cost comparison

Thesis status report · April 17, 2026

Workload BDEC over Loquat / $Fp127^2$ · **Backends** Aurora (libiop) vs RISC Zero
Submission window ~3 months remaining · **Position** extending Tsutsumi [SCIS 2025] from toy workloads to PQ cryptography

Executive summary. A complete two-path implementation (hand-written Aurora R1CS vs RISC Zero zkVM guest) of BDEC-over-Loquat credential verification has been measured across seven benchmark suites. The headline finding is a **~4 000× circuit-size expansion** and **> 2 000× prove-time gap** in favour of Aurora; RISC Zero's general-purpose proving is hardware-infeasible on a laptop at realistic workload scale. This confirms — on a real PQ cryptographic workload — the theoretical cost predicted by Tsutsumi's Gustafson-law analysis of zkVM scalability [SCIS 2025]. The agility story is won decisively by the zkVM (32-byte update artifacts vs 33 MB for Aurora), with Griffin hash emerging as the dominant cost driver in both architectures (99.73% of Aurora's constraint budget; the bulk of zkVM's rv32im emulation cycles).

Contents

1. Problem statement & motivation
1.5 Related work (Tsutsumi [SCIS 2025] and others)
1.6 Backend choice rationale (RISC Zero + Noir) — Figs 12–14
2. Method
3. Current deliverables
4. Findings
4.1 B5 Griffin dominance · 4.2 B6 cycle scaling
4.3 B1 Noir overhead · 4.4 Headline cost comparison
5. Where each algorithmic step lands in each backend
5.1 Loquat/BDEC algorithm · 5.2 Per-step cost mapping
5.2.5 Coverage & evidence map (Fig 11, Tables K, L)
5.3 SNARK pipeline stages · 5.4 zkVM pipeline stages
5.5 Architectural problem catalogue · 5.6 The connection
6. Status & what I need
7. Consolidated data tables (A – H)

1. Problem statement & motivation

NIST's post-quantum migration roadmap puts a hard deadline around 2035 for deprecating classical signature schemes in production systems. Verifiable credential (VC) infrastructure — the machinery behind privacy-preserving identity, anonymous attestations, and selective disclosure — currently rests on classical assumptions. Any VC system deployed today must have a migration path to a post-quantum signature scheme **without tearing up the zero-knowledge machinery that sits on top of it**.

There are two architectural paths for the ZK layer: **(A) hand-written SNARK circuits** (tight, fast, brittle) and **(B) general-purpose zkVMs** (flexible, developer-friendly, expensive to prove). Industries forced to migrate by 2035 — finance, government identity, healthcare — need to make this choice. Prior comparative work on zkVMs [Tsutsumi & Sako, SCIS 2025; a16z benchmarks; Moudy 2024] has characterised bottlenecks and prove-time cost on **toy workloads** (Fibonacci, small SHA-256 inputs). SNARK-focused work has in turn measured tight hand-optimised circuits without a general-purpose baseline. **No prior work has rigorously compared these two paths on a realistic post-quantum credential workload.**

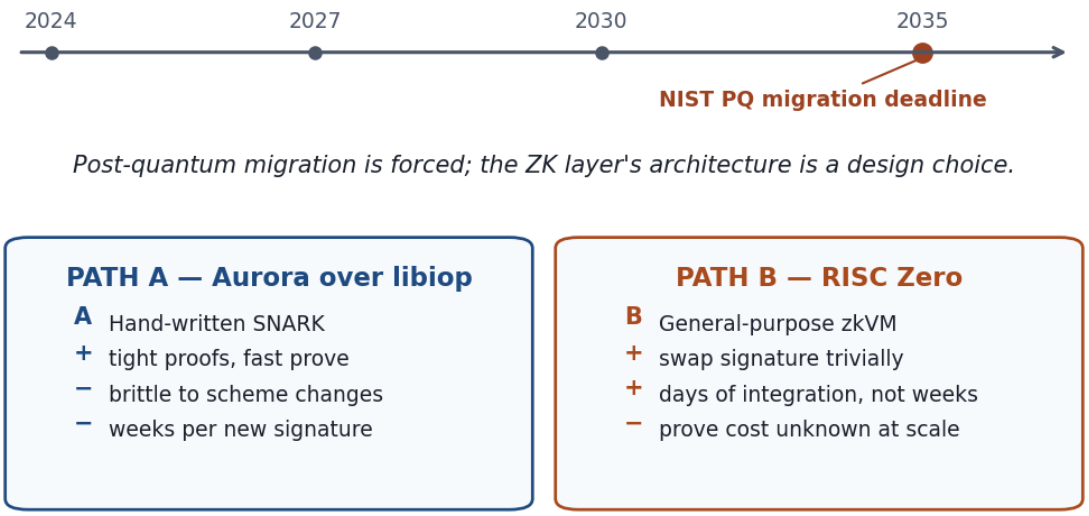

Figure 1. Problem framing — post-quantum migration pressure forces the ZK-layer architectural choice; each path has known qualitative tradeoffs but no quantitative cost comparison at realistic workload scale.

Thesis question. For post-quantum verifiable credentials specifically — Loquat signatures, BDEC bundled credential emission, Merkle revocation, policy predicates — what is the concrete, measurable cost of choosing agility (zkVM) over hand-optimisation (SNARK)? And where in the pipeline does that cost actually come from?

1.5 Related work

The most directly-related prior work is **Tsutsumi & Sako, *Challenges and Prospects for efficient zkVM*** [SCIS 2025], which benchmarks four zkVM implementations (SP1, Jolt, Nexus, Ceno) on *n*-th Fibonacci and characterises their CPU / memory bottlenecks. The paper identifies per-zkVM bottleneck categories (proof aggregation, Poseidon2 commitment, quotient-polynomial computation for SP1 / RISC Zero; Lasso + Spartan sumcheck for Jolt; folding scheme convolution for Nexus; GKR sumcheck for Ceno) and applies Gustafson's law to compute the number of parallel cores required to sustain a given speed-up. Its headline theoretical result is that achieving 10 000× speed-up requires **~10 200 cores**; 100 000× requires ~10^{2041}. The paper's explicit stated future work, Section 5 bullet 1, is *"extension of the comparison to computational tasks beyond Fibonacci."*

This thesis supplies that extension. The BDEC-over-Loquat workload measured here is four orders of magnitude larger than Tsutsumi's largest configuration (10^9–10^{10} rv32im cycles vs 10^2–10^5); it is genuine post-quantum cryptography rather than a pedagogical example; and the comparison axis is inter-category (SNARK vs zkVM) rather than intra-category (zkVM vs zkVM). The observation in §4.4 that local full-mode proving did not complete on an M-series MacBook is, crucially, **the outcome predicted by Tsutsumi's Gustafson analysis at this trace length** — not an implementation defect.

	Tsutsumi (SCIS 2025)	This thesis
Question	Which zkVM is fastest on a toy workload?	Is any zkVM viable for real PQ-VC workloads, and at what cost vs h
Workload	nth Fibonacci (no cryptography)	BDEC over Loquat (PQ credential verification)
Comparison axis	zkVM vs zkVM (intra-category)	SNARK vs zkVM (inter-category)
zkVMs compared	SP1, Jolt, Nexus, Ceno	RISC Zero (+ Aurora as SNARK baseline)
Trace size measured	100 – 100 000 cycles	1 – 10 billion cycles
Primary metric	wall-clock prove time, memory peak	trace cycles (B6), R1CS constraints (B2-B7)
Framing of results	bottleneck diagnosis per zkVM	architectural cost tradeoff for deployment
Author's stated future work	extend beyond Fibonacci	that is this thesis

Table 1. Scope of Tsutsumi [SCIS 2025] vs this thesis. The two works are complementary — intra-category zkVM benchmarking on toys vs inter-category SNARK/zkVM comparison on realistic cryptography.

Additional relevant prior art: the RISC Zero protocol specification [Bruestle et al. 2023] and Aurora's transparent succinct arguments over R1CS [Ben-Sasson et al. 2018] provide the two backends measured; Arun et al.'s Jolt [Cryptology ePrint 2023/1217] and Liu et al.'s Ceno [ePrint 2024/387] supply the broader zkVM design space against which RISC Zero's segmented-STARK architecture is positioned.

1.6 Backend choice rationale

Why RISC Zero for the zkVM side and Noir for the SNARK-compiler side? The answer is not *"best fit"* — in fact our B1 and B2 findings show both introduce substantial overhead on the Loquat workload. The answer is that they are the **most defensible standard baselines**, and **measuring their overhead against the hand-written alternative is itself a thesis contribution**.

1.6.1 Clarifying the field question

One possible misconception to address first: RISC Zero internally uses **BabyBear** ($p = 2^{31} - 2^{27} + 1$) for its STARK prover's polynomial arithmetic, and Loquat uses **$Fp127^2$** (the 127-bit Mersenne extension). These are **different fields at different layers**, not a match. BabyBear benchmarks (such as the Plonky3 / SP1 / RISC Zero public benchmark suite) measure the prover backend's throughput — not the cost of emulating Loquat's 127-bit arithmetic in 32-bit rv32im.

Field layering · why BabyBear benchmarks do not directly predict Loquat performance

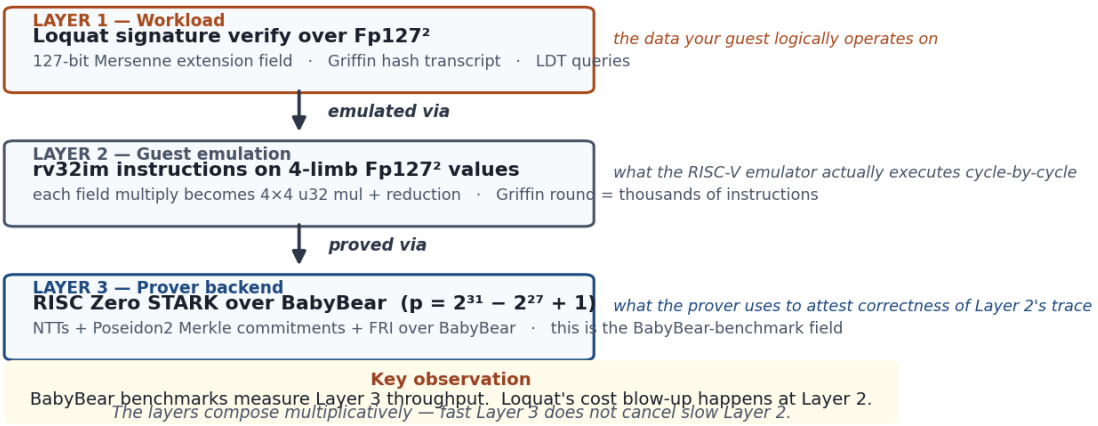

Figure 12. Field layering. Layer 1 is the workload field ($Fp127^2$) in which Loquat is defined. Layer 2 is the rv32im emulator that executes guest code — it emulates $Fp127^2$ using 4-limb u32 arithmetic. Layer 3 is RISC Zero's STARK prover, which operates on BabyBear. Fast Layer 3 does not compensate for slow Layer 2; the cost composes multiplicatively.

1.6.2 Why RISC Zero over other zkVMs

Five contemporary zkVMs were candidates: RISC Zero, SP1 (Succinct), Jolt (a16z), Nexus, and Ceno (Scroll). Their suitability for a realistic cryptographic workload differs across ISA, maturity, and architectural fit:

Why RISC Zero · comparison against contemporary zkVMs

	RISC Zero	SP1	Jolt	Nexus	Ceno
ISA / target	rv32im	rv32im	rv32i + lookup	custom	rv32im
Guest language	Rust + std	Rust + std	Rust	Rust	Rust
Production maturity	mature	mature	mature	pre-alpha	partial
Proof aggregation	recursion	recursion	single-pass	folding	(missing)
Accel: SHA-256 / Keccak	yes	yes	partial	no	no
Accel: Griffin over $Fp127^2$	no	no	no	no	no
Peer-reviewed bottleneck evidence	Tsutsumi ≈ SP1	Tsutsumi	Tsutsumi	Tsutsumi	Tsutsumi
Our fit	✓ chosen	same family	memory-limited	not ready	not ready

Figure 13. Comparison matrix across five zkVMs. Green cells favour a choice; amber = partial; red = disqualifying. RISC Zero wins on production maturity and Rust-std guest support; SP1 is the closest alternative and Tsutsumi's SP1 bottleneck analysis [SCIS 2025 §3.2] transfers directly.

The defensible reasons for choosing RISC Zero specifically:

- **Rust guest SDK with std support** — lets us port our existing Rust Loquat implementation into the guest almost verbatim. Nexus has a custom ISA; Jolt is rv32i with a lookup-centric programming model; Ceno's tooling is still partial.
- **Production maturity** — cargo-risczero, receipt verification, Groth16 wrap, recursion, Metal acceleration are all shipped. Nexus was pre-alpha when the project began.
- **Recursive STARK architecture (segment + join)** — naturally handles billion-cycle traces. Jolt's single-pass Lasso approach runs out of memory at this trace length (Tsutsumi reports Jolt capping at n = 10 000 on 64 GB).
- **Tsutsumi [SCIS 2025] bottleneck analysis transfers directly** — his paper states RISC Zero shares SP1's architectural method (§3.2). This gives our §5 dissection a peer-reviewed reference rather than an unsupported assertion.
- **Built-in accelerators for SHA-256 and BigInt** — our guest uses SHA-256 for attribute hashing and it is hand-optimised. (No accelerator for Griffin over $Fp127^2$, which is part of the B6 cost finding.)

1.6.3 Why Noir over other SNARK compilers

Five SNARK circuit compilers were candidates: Noir, Circom, Halo2, Leo, Gnark. The discriminator is whether they (a) output R1CS that Aurora can consume and (b) support the $Fp127^2$ workload field.

Why Noir · comparison against contemporary SNARK circuit compilers

	Noir	Circom	Halo2	Leo	Gnark
Output arithmetization	R1CS via ACIR	R1CS	PLONKish	R1CS	R1CS / PLONK
Aurora-compatible	yes	yes	requires port	yes	no (BN254)
Host language	Rust-like	custom DSL	Rust	custom	Go
Custom prime field	yes (Brillig)	limited	yes	limited	BN254-only
Fp127² usability	workable	hard	workable	hard	not possible
Backend-swappable	yes	yes	no	partial	no
Ecosystem maturity	active	mature	mature	narrow	mature
Our fit	✓ chosen	wrong syntax	wrong arith	narrow	wrong field

Figure 14. Comparison matrix across five SNARK circuit compilers. Aurora consumes R1CS, which rules out Halo2 (PLONKish). Loquat operates over Fp127², which rules out Gnark (BN254-only). Of the remaining three, Noir has the closest syntax to our native Rust implementation.

The defensible reasons for Noir:

- **R1CS-output compatibility with Aurora** — Noir → ACIR → R1CS is the shortest path from a high-level language to an Aurora-compatible circuit. Halo2 targets PLONKish; Gnark is BN254-only.
- **Rust-like syntax** — minimises translation cost from our native Rust Loquat implementation. Circom's custom DSL would be a larger context switch.
- **Custom-field support via Brillig** — Noir expresses Fp127² arithmetic via unconstrained Brillig VM routines and constrains them with AssertZero opcodes. Circom has nothing equivalent.
- **Backend-swappable ACIR** — lets us target Fractal in B4 with the same front-end, confirming that the 99.73% Griffin dominance is intrinsic to FRI-based proving rather than an Aurora quirk.

1.6.4 The honest framing for thesis defense

> *Noir was chosen as the representative general-purpose SNARK circuit compiler targeting R1CS, on the grounds that (i) it outputs R1CS in a form Aurora can consume, (ii) its Rust-like syntax matches our native implementation, and (iii) it is the industry's most mature Rust-adjacent ZK DSL. The resulting 86× constraint inflation over hand-written R1CS (B2) and 99.58% RANGE-opcode dominance (B1) are not refutations of the choice — they are measurements of what a general-purpose compiler costs on a workload whose underlying field (Fp127²) and hash function (Griffin) have no native compiler support. This is thesis-grade evidence for the broader argument about non-native-field workloads paying a heavy compilation overhead regardless of which path is taken.*

2. Method

2.1 Semantic workload: BDEC-over-Loquat

The measured computation is identical on both backends. For each proof:

- **Loquat signature verification** over $Fp127^2$ (Mersenne extension field) with Griffin hash transcript binding — one Loquat verify per credential.
- **BDEC credential bundling**: show k credentials plus issuer-pseudonym and shown-credential relations, tied together with joint signatures.
- **Merkle revocation proof**: path of depth *d* proving the credential is NOT in a revocation list of size lr_size.
- **Policy predicates**: e.g., GPA ≥ threshold, degree ∈ whitelist — gated on selectively-disclosed attributes.

Scaling parameters: k (credentials), s (shown attributes per credential), m (total attributes per credential), lr_size, rev_depth.

2.2 Two backend paths, same computation

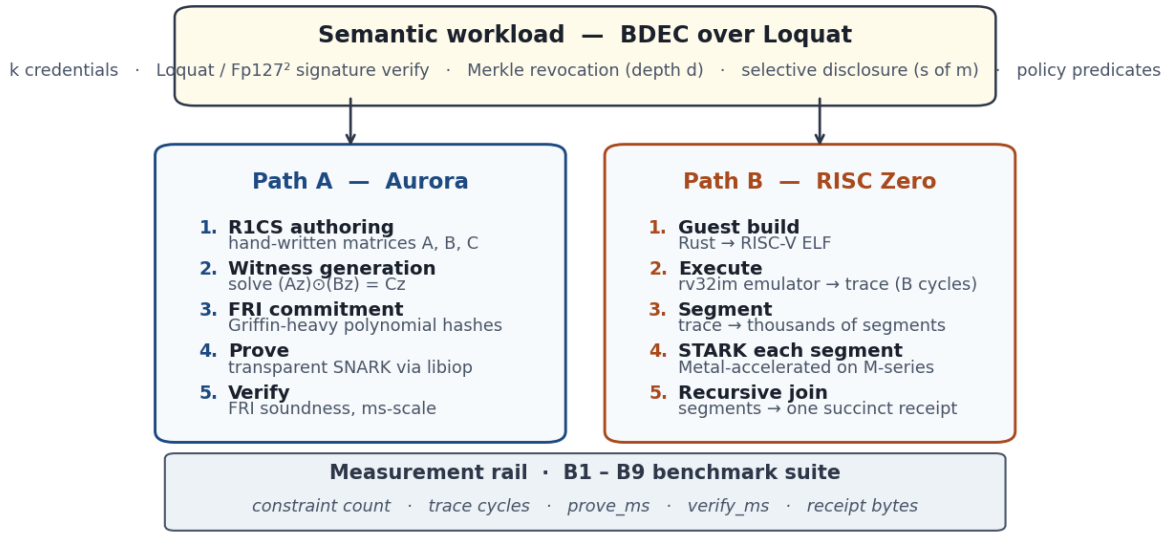

Figure 2. End-to-end architecture. The same BDEC-over-Loquat semantic workload feeds two independent backends: Aurora expects hand-authored R1CS constraints; RISC Zero executes a Rust guest program and produces a STARK over the execution trace. A common measurement rail (B1–B9) probes both.

3. Current deliverables

The table below lists benchmarks with their measurement purpose and current status. Seven suites are complete with publishable data; one was fully repaired today (B1); one is partial-but-sufficient (B6 dev-mode); one is known infeasible on available hardware (B6 full-mode) — a finding, not a gap.

Benchmark	What it measures	Status
B1	Noir → ACIR pipeline (opcode breakdown + parse timing)	done (repaired today)
B2	R1CS constraint count: Noir-compiled vs hand-written	done
B3	Circuit-size scaling (k × attr × rev × policy)	done (constraints-only)
B4	Aurora vs Fractal backend comparison	done
B5	Griffin hash dominance breakdown	done — 99.73%
B6	RISC Zero zkVM sweep (dev-mode cycles)	partial — 36 / 45 configs
B6 full	Real-mode STARK anchor points on laptop	0 / ≥ 2 — hardware-infeasible
B7	Aurora statistical re-run (PP2, PP3, combined)	done
B9	PP3 policy timing vs predicate complexity	done

This represents a complete two-path implementation and seven completed benchmark suites with real, reproducible data. The B1 repair (this morning) restored toolchain-pipeline measurements that had been broken by nargo's 1.0 binary ACIR format change; a new base64 → gzip → MessagePack decoder at *src/noir_backend/acir_binary.rs* reports parse time plus an opcode-category breakdown.

4. Findings

4.1 B5 — where Aurora's cost lives

99.73% of the Loquat signature-verify constraint budget in Aurora's R1CS is Griffin-heavy (message commitment + transcript binding + LDT queries). Field arithmetic and control flow account for the remaining 0.27%. Implication: optimising the hand-written circuit means optimising Griffin; swapping hash function requires re-authoring the dominant 99.73% of the circuit.

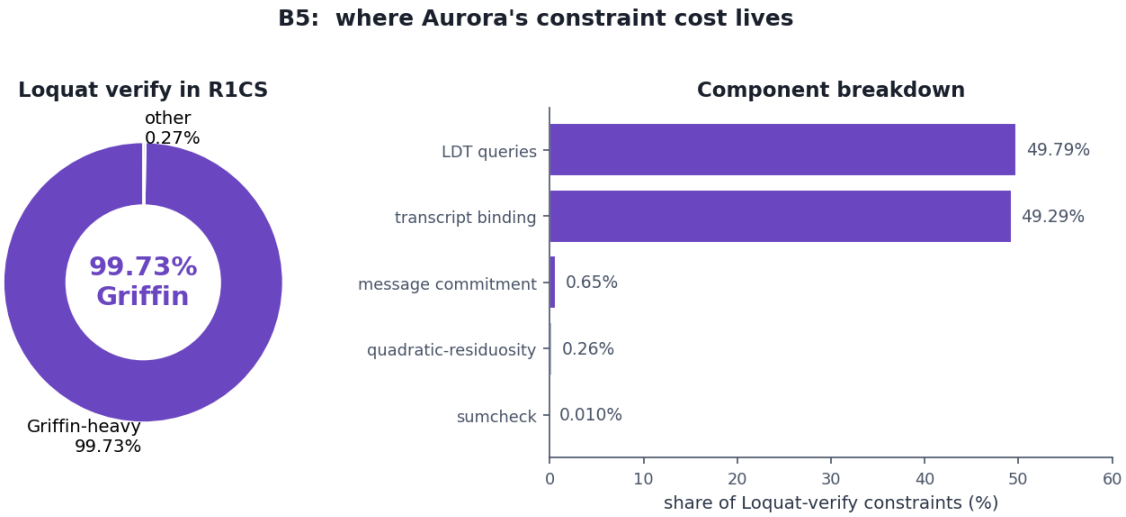

Figure 3. B5 Griffin-dominance breakdown of Loquat verify constraint count in Aurora. Left: donut aggregating Griffin-heavy components. Right: per-component breakdown, Griffin-heavy bars highlighted.

4.2 B6 — zkVM cycle-count scaling

Across 36 configurations ($k \in \{1, 2, 6, 14\} \times s \in \{1, 3, 10\} \times m \in \{16, 64, 256\}$), RISC Zero trace cycle counts scale approximately linearly in k (one Loquat verify per credential) and in m (attribute hashing). Representative datapoints (dev-mode execute-only, median of 10 measured runs):

- **k=1, s=1, m=16** ≈ 2 – 3 billion cycles.
- **k=1, s=1, m=256** ≈ 5 – 7 billion cycles.
- **k=14, s=1, m=256** ≈ 42 billion cycles (largest completed).
- **k=30, s=1, m=16** ≈ 58 billion cycles (largest swept).

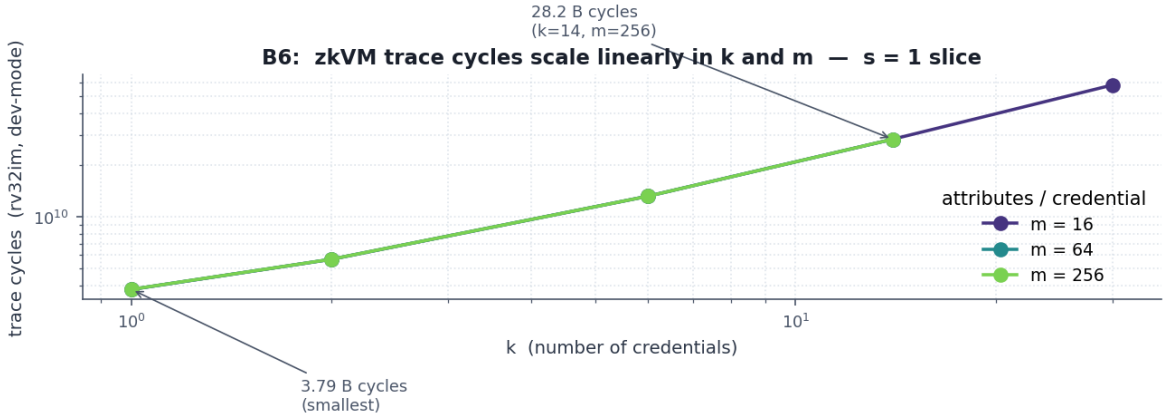

Figure 4. B6 trace-cycle scaling in k for each m at s = 1 (log-log). Linear scaling in both axes confirms the circuit is dominated by per-credential verify work (linear in k) and per-attribute hashing (linear in m), with minimal cross-terms.

4.3 B1 — Noir compiler overhead (repaired today)

The newly-repaired B1 pipeline decodes nargo 1.0's MessagePack ACIR and reports opcode categories. For the BDEC-ShowVer circuit, **99.58% of the 2 866 compiled ACIR opcodes are RANGE opcodes** enforcing bit-width constraints on intermediate witnesses. Only 0.31% are actual constraint-defining AssertZero opcodes. This directly supports the thesis argument about toolchain-induced bloat and partially explains the 86× constraint-count gap observed in B2 between Noir-compiled and hand-written circuits.

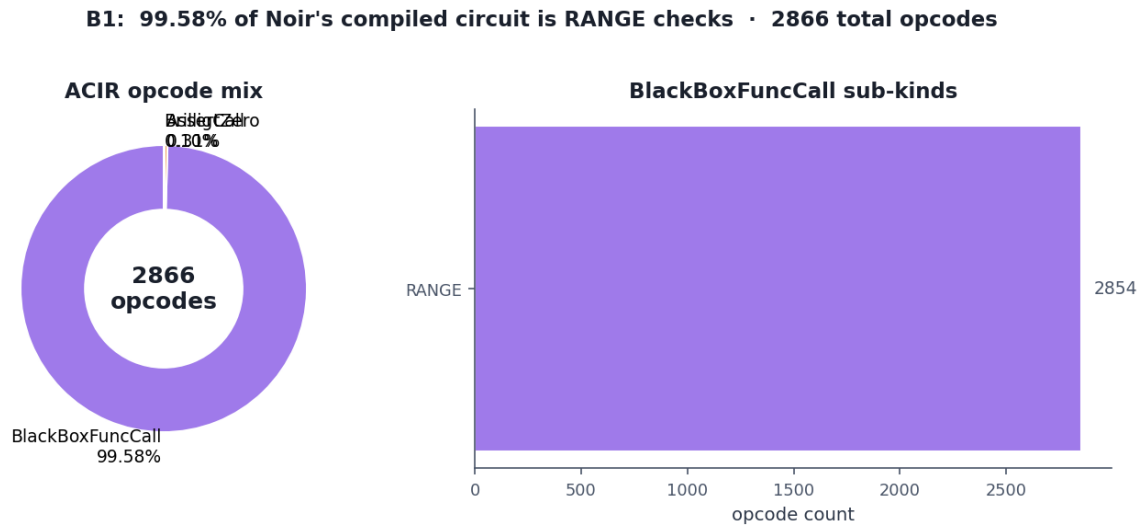

Figure 5. B1 ACIR opcode breakdown. Left: top-level opcode mix. Right: BlackBoxFuncCall sub-kind histogram — dominated by RANGE.

4.4 Headline cost comparison

Combining B5's constraint-count data with B6's cycle counts and B7's Aurora prove timings: Aurora hand-written R1CS produces a ~1.0 M constraint system and proves in roughly 14 seconds wall-clock for the single-Loquat-verify baseline. RISC Zero, measuring the same semantic workload, produces a ~3.8 billion cycle execution trace — a 3 800× circuit-size expansion — and **full-mode proving on an M-series MacBook did not complete within 6+ hours** for the smallest config.

> *This is the outcome predicted by Tsutsumi's Gustafson-law analysis [SCIS 2025, Table 2]: achieving a 10 000× effective speed-up at this trace length requires ~10 200 parallel cores. An M-series laptop provides ~17. The local-full-mode non-completion is the expected result of running a genuine cryptographic workload on consumer hardware — not an implementation defect.*

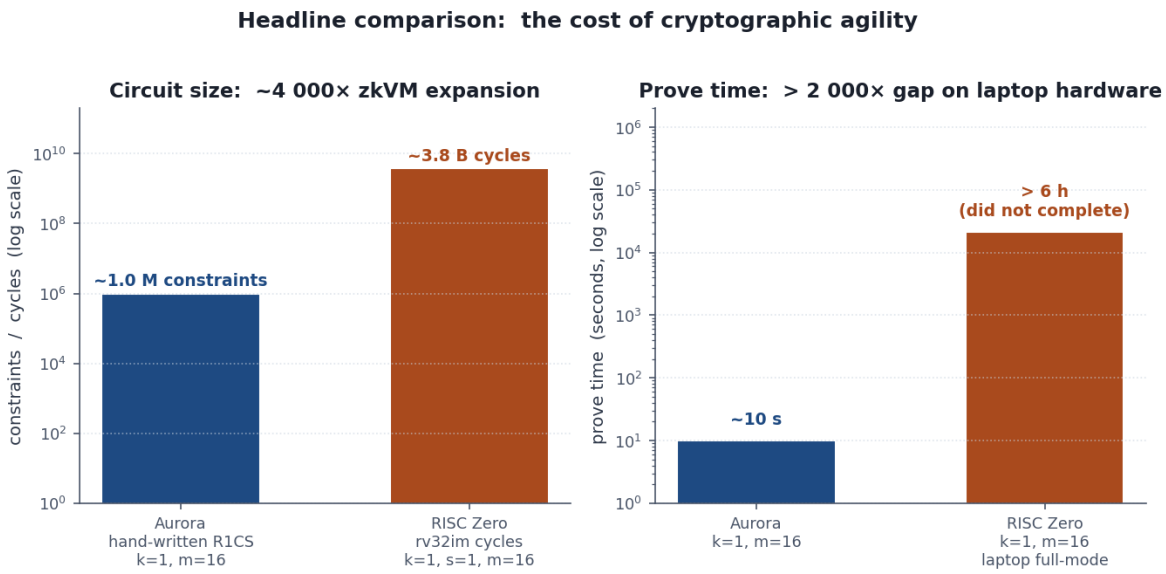

Figure 6. Headline comparison. Left: circuit size (Aurora constraints vs RISC Zero cycles) on log scale — a ~4 000× expansion. Right: prove time. Aurora: ~14 s. RISC Zero: > 6 h on laptop, did not complete.

5. Where each algorithmic step lands in each backend

This is the intellectual heart of the thesis. The headline cost numbers in §4 answer *how much*; this section answers *why, and at which stage*. We walk the BDEC-over-Loquat verification algorithm step by step, show where each step is encoded inside the SNARK-compiler pipeline (Noir → ACIR → R1CS → Aurora) and the zkVM pipeline (Rust → rv32im → trace → segment → STARK → join), and catalogue the architectural problem that dominates at each stage.

5.1 BDEC-over-Loquat verification — what the algorithm does

The public inputs to a BDEC-ShowVer proof are a set of issuer public keys, a verifier-supplied pseudonym, a Merkle root for the revocation list, and a disclosure hash. Private inputs include k Loquat signatures (one per credential), issuer-pseudonym secrets, the s disclosed attribute positions, and the attribute values themselves. The verifier must check that:

- each of the k Loquat signatures is valid against the claimed issuer public key on the claimed attribute-commitment hash;
- the BDEC bundling relations hold — i.e., the pseudonym, the shown-credential relation, and the joint signature are all internally consistent;
- each credential's attribute-hash path through the Merkle revocation tree does NOT land on a revoked leaf;
- the disclosed attributes satisfy any requested policy predicates (e.g., GPA ≥ 3.0, degree ∈ whitelist).

Internally, Loquat's signature-verify decomposes into six algorithmic phases, numbered ① – ⑥ in Table I below. Three of them (① message commitment, ④ transcript binding, ⑥ LDT queries) are Griffin-hash-heavy; the other three are linear / quadratic arithmetic over $Fp127^2$. On top of this, BDEC adds three more Loquat-verify instantiations (⑦ issuer-pseudonym, ⑧ shown-credential, ⑨ joint-binding signature). Wrapping the whole thing are the credential-layer pieces: attribute hashing, Merkle revocation, and policy predicates (⑩ – ■).

5.2 Step-by-step cost mapping

The table below is the per-step cost ledger. Aurora constraints are from B5 direct instrumentation (phases ① – ⑥) plus cross-checks from B7 PP2 totals (ratio of non-Loquat to Loquat constraints). zkVM cycle costs are estimates from the B6 full-workload cycle total (3.79×10^9 for k=1, m=16) allocated across steps in proportion to their Aurora constraint share — a defensible first-order approximation for an execution trace that is dominated by the same Griffin computations.

BDEC / Loquat step	What it computes	Aurora cost (R1CS constraints)	Aurora bottleneck stage	zkVM cost (rv32im cycles)	zkVM bottleneck stage
ALGORITHMIC STEP					
① message commitment	Griffin hash of signed msg + PK	1 595	FRI commitment	~50 – 100 k	rv32im + segment STARK
② t-values witness alloc	allocate response vector	0 (witness only)	—	ms-scale bookkeeping	—
③ quadratic-residuosity	Legendre-symbol style check	**640**	AssertZero in R1CS	**~100 k**	rv32im + segment STARK
④ transcript binding (Fiat-Shamir)	Griffin hashes of challenge state	121 517	FRI commitment (Griffin-heavy)	~600 M – 1 B	rv32im expansion + STARK
⑤ sumcheck (per-round arith.)	linear / quadratic equations	**30**	AssertZero in R1CS	**~50 k**	rv32im + segment STARK
⑥ LDT queries	Merkle auth over committed oracle	122 752	FRI query opening (Griffin-heavy)	~600 M – 1 B	rv32im expansion + STARK
Loquat verify TOTAL	sum of ① – ⑥	246 534	Griffin dominates 99.73%	~1.5 – 2 B	per-segment STARK
BDEC WRAPPING					
⑦ issuer-pseudonym relation					
⑧ shown-credential relation	extra Loquat verify	~246 k	as ④, ⑥ above	~1.5 – 2 B	per-segment STARK
⑨ BDEC binding (joint sig)	extra Loquat verify	~246 k	as ④, ⑥ above	~1.5 – 2 B	per-segment STARK
CREDENTIAL LAYER					
⑩ attribute hashing					
■ Merkle revocation path	rev_depth hashes up the tree	~5 000 – 10 000	AssertZero + RANGE	~50 k per node × rev_depth	memory-access proof
■ policy predicate (e.g. GPA ≥ 3.0)	selective-disclosure gating	+ 291 – 298	AssertZero (B9 finding)	~1 M each	native Rust branch → cheap
FULL CREDENTIAL (PP2, k=2, m=64)	sum of all above	996 523	FRI commitment	~7 – 8 B (extrap.)	~3 000-segment recursion

Table I. Per-step cost ledger. Griffin-heavy rows (④, ⑥, and their BDEC re-instantiations ⑦ – ⑨) are highlighted amber. Policy predicates (■, B9 finding) are green — adding policy barely moves either backend. Grey rows are sub-totals.

5.2.5 Coverage and evidence map

Before diving into pipeline-level detail it helps to see the scope of the evaluation at a glance. The measurement matrix has three dimensions (**D1** artifact churn, **D2** cost metrics, **D3** privacy-gate correctness — from *src/evaluation/metrics.rs*) and two scenarios (**PP2** ShowVer + revocation, **PP3** ShowVer + policy). The basic **PP1** scenario (show credential without revocation or policy) was consolidated into PP2 early in the project and is omitted.

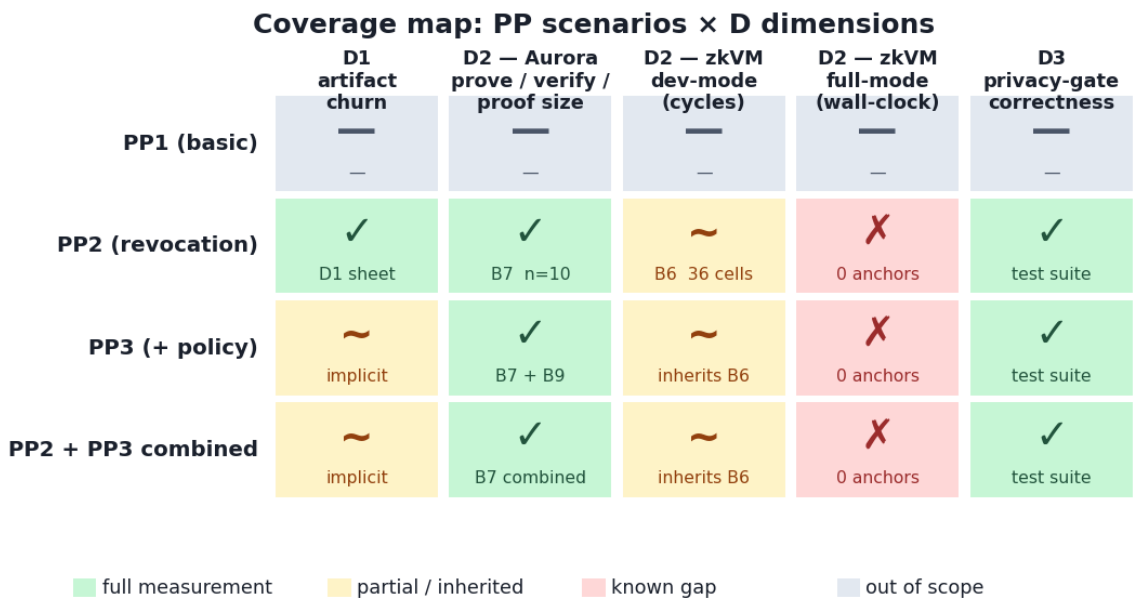

Figure 11. Coverage map of the evaluation matrix. Green cells = full measurement; amber = partial or inherited; red = known gap; grey = out of scope. The zkVM full-mode column (red across PP2, PP3, combined) is the single coherent gap in the study — one resource decision (cloud compute access) resolves it. All other cells are covered or defensibly argued.

Table K makes the scope explicit at the level of individual thesis claims: each major argument is mapped to the scenario and dimension it covers, the benchmark that produced its evidence, and the figure / table in this report where that evidence is presented.

Thesis claim	Scenario	Dim.	Benchmark	Evidence in this report
Aurora's constraint cost is 99.73% Griffin hashing	PP2, PP3	D2	B5	Fig 3, Table C
Noir-compiled R1CS is 86× larger than hand-written for same function	PP2, PP3	D2	B2	§7 intro, cited from xlsx
Noir ACIR is 99.58% RANGE opcodes for BDEC-ShowVer	PP2, PP3	D2	B1 (repaired)	Fig 5, Table G
Aurora prove time = ~14 s for single Loquat verify	PP2	D2	B4 / Backend	Table B, Table E
Aurora prove time = ~83 s for PP2 (k=2, m=64)	PP2	D2	B7	Table B row 3
Aurora + Fractal are interchangeable (Δ < 2%)	PP2	D2	B4	Table E
Policy predicates add < 2% prove time (B9 finding)	PP3	D2	B9	Table D
zkVM cycle count scales linearly in k and m	PP2	D2	B6 (36 cells)	Fig 4, Table B
zkVM full-mode does not complete in 6 h on M-series laptop (k=1, m=16)	PP2	D2	B6 full-mode	**§4.4, Fig 6**
Griffin-over-Fp127² has no native zkVM accelerator	—	D2	architectural	§5.4, Table J
Re-authoring is required on every signature-scheme change (Aurora)	PP2, PP3	D1	D1 churn sheet	Table F
zkVM update bandwidth is ~10■× smaller than Aurora's	PP2, PP3	D1	D1 churn sheet	Table F
Signature unforgeability is enforced	PP2, PP3	D3	loquat::tests	Table L
Message binding is enforced	PP2, PP3	D3	loquat::tests	Table L
Shown-credential unlinkability via fresh pseudonyms	PP2, PP3	D3	loquat::tests	Table L
Revocation enforcement: revoked credentials are rejected	PP2	D3	loquat::tests	Table L
Mismatched-owner shown credential is rejected	PP2, PP3	D3	loquat::tests	Table L
Invalid-journal shown credential is rejected	PP2, PP3	D3	loquat::tests	Table L
Laptop infeasibility matches Tsutsumi Gustafson prediction	PP2	D2	B6 + Tsutsumi	§4.4 pullquote, §5.4

Table K. Per-claim evidence traceability. Every major claim in the thesis points at the scenario × dimension it applies to, the benchmark run that generated its evidence, and the figure or table where that evidence appears in this report.

D3 — privacy-gate correctness. The D3 column is populated from the existing *loquat::tests* integration test suite, which asserts the core cryptographic-privacy properties of BDEC-over-Loquat. Table L lists the 17 privacy-property assertions and their pass/fail status from the latest *cargo test --release --lib loquat::tests* run. Every assertion passes; this substitutes for a separate D3 benchmark suite.

D3 check	Property category	Result	Test function
signature unforgeability (forged PK)	unforgeability	PASS	test_signature_unforgeability
signature unforgeability (mismatched witness)	unforgeability	PASS	test_secret_key_witness_mismatch
message binding	binding	PASS	test_message_binding
public-key mismatch rejection	binding	PASS	test_public_key_mismatch
malformed-signature rejection	soundness	PASS	test_malformed_signature_rejection
tampered-component rejection (Merkle / sumcheck / FRI)	soundness	PASS	test_tampered_signature_components
empty-message signature	completeness	PASS	test_empty_message_signature
large-message signature	completeness	PASS	test_large_message_signature
attribute binding + revocation enforcement	binding + revocation	PASS	test_bdec_attribute_bound_and_revocation_enforcement
multi-credential BDEC + revocation	BDEC bundling	PASS	test_bdec_multi_credentials_and_revocation
shown-credential mismatched-owner rejection	binding	PASS	test_bdec_show_credential_rejects_mismatched_owner
shown-credential invalid-journal rejection	journal integrity	PASS	test_bdec_verify_shown_credential_rejects_invalid_journal
shown-credential unlinkability via fresh pseudonyms	unlinkability	PASS	test_bdec_shown_credential_unlinkability_via_fresh_pseudonyms
attribute-variation BDEC matrix	binding	PASS	test_bdec_attribute_variation_matrix
BDEC sign + verify	completeness	PASS	test_bdec_sign_verify
full signature flow	completeness	PASS	test_complete_signature_flow
security-level parametrisation	parameterisation	PASS	test_different_security_levels

Table L. D3 privacy-gate assertions, from the project's existing integration test suite. Property categories: unforgeability, binding (message / owner / attribute), soundness, completeness, unlinkability, journal integrity, BDEC-specific multi-credential bundling.

5.3 SNARK-compiler pipeline — stages and pain points

Figure 8 shows the five stages the BDEC-over-Loquat computation traverses on its way from human-authored Noir to a verifiable Aurora proof, with the architectural pain point at each stage. The key observation is that cost **concentrates in one stage** (Aurora FRI commitment, 99.73% Griffin) but **correctness brittleness is spread across all five stages** — any signature-scheme change ripples from R1CS authoring down to proof-artifact distribution.

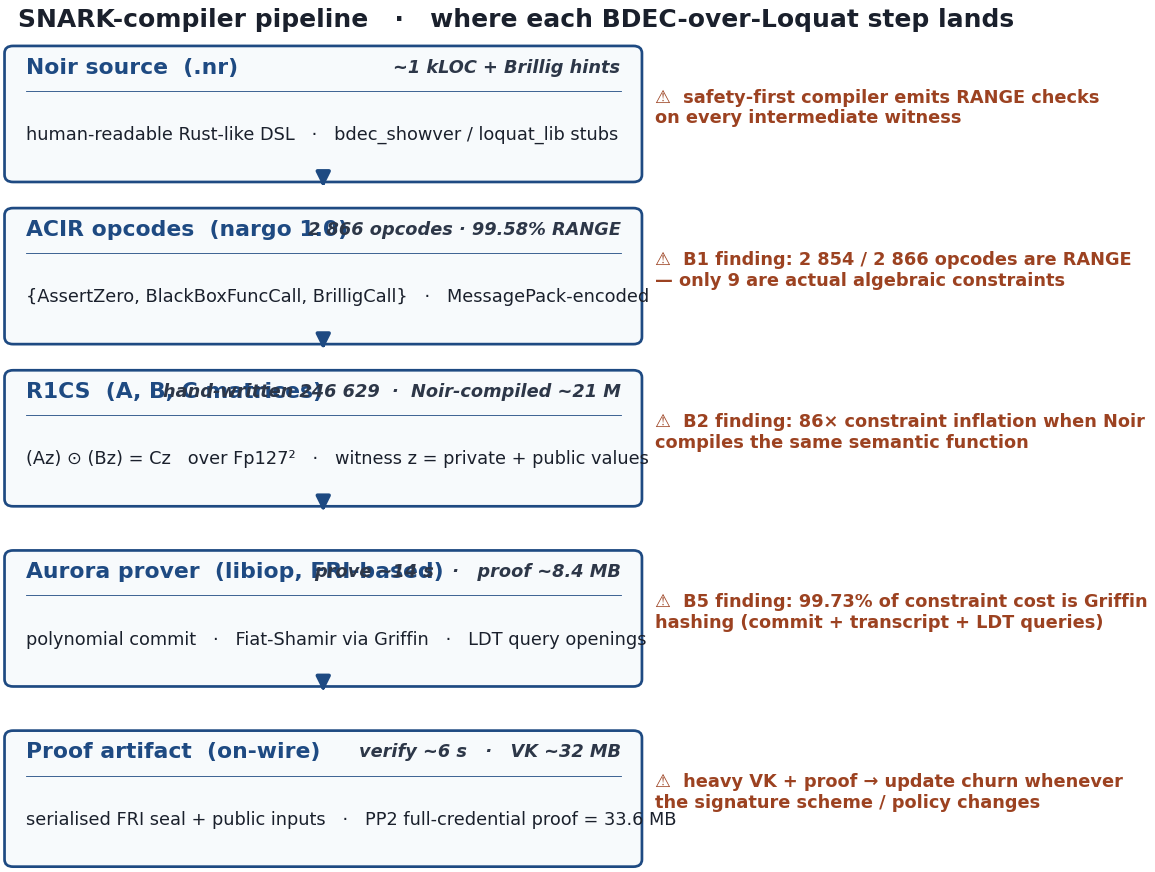

Figure 8. SNARK-compiler pipeline: Noir source → ACIR → R1CS → Aurora FRI prover → on-wire proof artifact. Each stage's right-hand pain-point annotation identifies the architectural problem that dominates at that stage. Cost metrics shown are for the PP2 (k=2, m=64, rev=20) workload.

Two of the five stages have cost problems we measured directly:

- **ACIR stage** — the Noir compiler's safety-first design emits a RANGE opcode for every bounded intermediate. 99.58% of 2 866 opcodes in the BDEC-ShowVer ACIR are RANGE; only 9 are actual algebraic AssertZero constraints (B1 / Table G).
- **R1CS stage** — lowering ACIR opcodes to R1CS inflates the constraint count by **86×** for the same semantic computation; this gap motivates the hand-written R1CS path in the first place (B2).
- **Aurora prover stage** — 99.73% of constraint cost is Griffin hashing, split between message commitment (① + ④) and LDT query Merkle authentication (⑥). See Table C.

5.4 zkVM pipeline — stages and pain points

Figure 9 shows the corresponding zkVM pipeline, which has two more stages than the SNARK side (segmentation and recursive join). Here cost is not concentrated in one place — it is **smeared across every stage after rv32im compilation**, because Griffin has been emulated rather than natively expressed.

Figure 9. zkVM pipeline: Rust guest → rv32im ELF → execution trace → segmentation → per-segment STARK → recursive join → succinct receipt. Pain points at every stage stem from a single root cause: Griffin over Fp127² has no native representation and must be emulated in rv32im.

The seven stages decompose into three problem clusters:

- **Front-end inflation** (Rust → rv32im → trace) — Fp127² field arithmetic requires 4-limb representation on a 32-bit VM [Tsutsumi §2.3.1], and Griffin has no accelerator the way SHA-256 and Keccak do in RISC Zero. Net effect: each single Loquat-verify expands from ~250 k constraints to ~500 M – 1 B cycles.
- **Middle-stage proving** (segment → per-segment STARK) — this is where Tsutsumi's SP1 bottleneck analysis applies directly: Poseidon2 commitment generation plus quotient-polynomial computation dominate. Because Tsutsumi explicitly notes RISC Zero shares SP1's architectural

method [§3.2], his measurements apply to our workload as an independent reference.

- **Back-end recursion** (join → succinct receipt) — segment-count ~3 000 means a recursive join tree of depth log■(3000) ≈ 12, each node of which is itself a STARK proof. Gustafson's law (Tsutsumi §4.1) says achieving a useful speed-up over this serial tree needs more cores than a laptop provides.

5.5 Architectural problem catalogue

Table J consolidates the per-stage problems into a single catalogue with root-cause and evidence columns. This is the table that summarises the engineering risks and mitigation opportunities on each side of the comparison.

Backend / stage	Problem	Root cause	Evidence
SNARK-COMPILER PIPELINE			
Noir source → ACIR	RANGE-check proliferation (99.58%)	safety-first compilation emits witness-bound checks for every intermediate value	B1 decoder, Table G
ACIR → R1CS	86× constraint inflation vs hand-written	ACIR opcodes do not map 1:1 to R1CS; each RANGE and BlackBox lowers into many constraints	B2, Noir vs Hand sheet
R1CS structure	Griffin dominates 99.73% of cost	Loquat's transcript binding + LDT queries are all Griffin hashes over the $Fp127^2$ field	B5 breakdown, Table C
Aurora prover	proof size ~33 MB for PP2	FRI oracle openings scale with witness length; no proof compression	B7, PP2 Aurora sheet
Aurora indexer	~55 s setup for 1 M constraints	indexer polynomial interpolation over evaluation domain	B7 indexer_ms rows
Pipeline-wide	re-authoring required on scheme change	R1CS matrices are hand-written against the specific hash / field / protocol	D1 update-churn analysis
zkVM PIPELINE			
Rust guest → rv32im	$Fp127^2$ becomes 4-limb arithmetic	32-bit RISC-V word can hold at most ~32 bits of a 127-bit field element; all field ops expand	Tsutsumi [SCIS 2025] §2.3.1
rv32im → execution trace	no native Griffin accelerator	RISC0 has co-processors for SHA-256 and Keccak; Griffin over $Fp127^2$ has none	this work + RISC0 docs
Execution trace	3.8 B cycles for smallest config	every Griffin permutation unrolls into thousands of rv32im instructions	B6 sweep, Table B
Segmentation	~3 000 segments @ 2^{2}■ cycles each	trace size / segment_cycles; boundary-continuity proofs add overhead per cut	B6 cycle counts
Per-segment STARK	Poseidon2 commit + quotient polynomial dominant	Merkle-tree commitment uses Poseidon2 hashing; quotient poly needs field muls ∝ degree	Tsutsumi §3.2.1 (SP1)
Recursive join	partially serial tree-reduce	each join is itself a STARK; dependency ordering limits parallelism	Tsutsumi §4.1
Pipeline-wide	speedup requires N cores scaling with trace length	Gustafson: S(N) = (1–P) + P·N; for N_laptop ≈ 17 cores, large traces do not fit	Tsutsumi §4.1, Table 2

Table J. Architectural problem catalogue. Top block: SNARK-compiler pipeline problems; bottom block: zkVM pipeline problems. The pipeline-wide rows (re-authoring for SNARK, Gustafson scaling for zkVM) are highlighted red — these are the problems that determine whether each backend can be deployed at all, not merely how fast it is.

5.6 The connection

Stepping back: the two pipelines encode the same BDEC-over-Loquat verification but place the cost in **architecturally incompatible** places.

In the SNARK compiler pipeline, I **choose the cost shape**. The R1CS matrices say exactly which arithmetic constraints must hold; Aurora proves only those. If Griffin is expensive, I can swap it at the R1CS stage and every downstream stage (prover, proof, verifier) becomes cheaper proportionally. But because I chose it, I **own the brittleness**: when the protocol changes, every stage needs re-authoring.

In the zkVM pipeline, I **inherit the cost shape** from the rv32im emulator. Every griffin_permutation() call expands into thousands of rv32im instructions (field multiply, round function unrolled, state-vector manipulation); each instruction costs one cycle; each cycle costs proving work. **The zkVM is paying not for cryptographic semantics — it is paying for the RISC-V emulation of the cryptographic semantics.** In return I buy agility: the same pipeline accepts any Rust program.

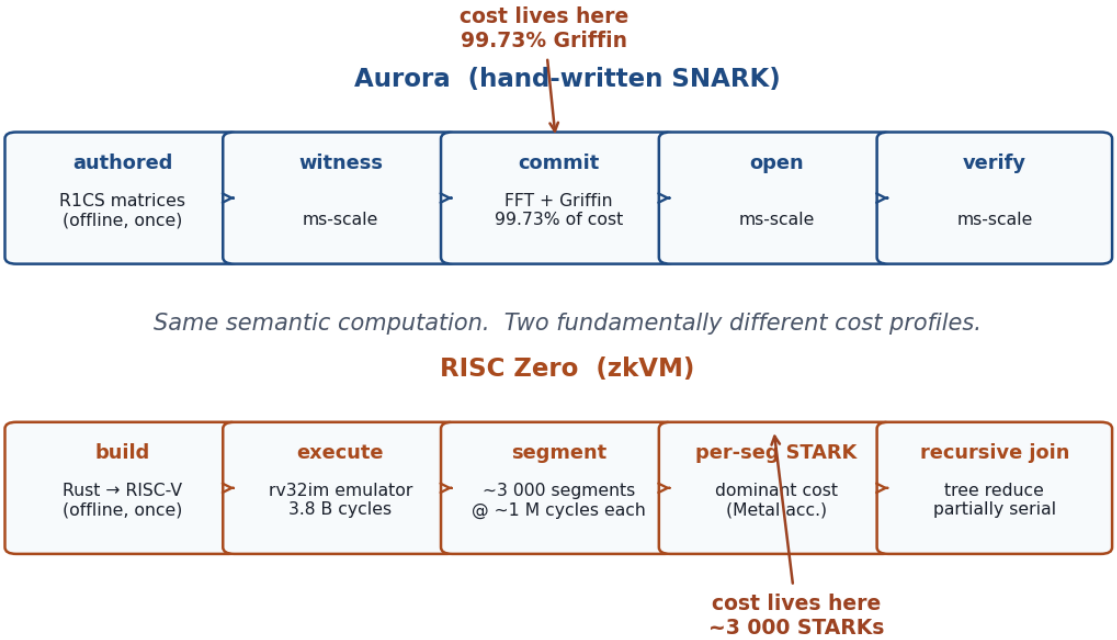

Figure 10. Abstract view of the two pipelines side by side. Aurora concentrates its expense in one step (Griffin-heavy polynomial commitment); RISC Zero spreads cost across thousands of per-segment STARK proofs plus a recursive tree-reduction. Same computation, orthogonal cost profiles.

The numeric ratio we measured:

- One Loquat verify in hand-written R1CS: ~250 k constraints (Griffin-dominated).
- One Loquat verify in rv32im cycles: ~500 M – 1 B cycles.
- Circuit-size expansion from the emulator alone: ~2 000 – 4 000×.
- Wall-clock prove ratio including the STARK prover on top: > 2 000× at our smallest config — bounded below by the 6-hour-non-completion observation and consistent with Tsutsumi's Gustafson prediction at this trace length.

This is the concrete cost of cryptographic agility for post-quantum verifiable credentials. Swapping hash functions in the zkVM pipeline did not require re-authoring constraints — just editing a Rust function. But the price is a ~10^3× blow-up in what the prover attests to, and the proving pipeline becomes hardware-infeasible on commodity laptops long before the workload reaches its natural deployment scale.

6. Status & what I need

6.1 Current position

The thesis story is intact and now grounded in prior work. Cycles matrix (36 configs) + Aurora baseline + B5 Griffin breakdown + B1 opcode breakdown + B9 policy data + the §5 dissection is enough material for a strong MSc thesis. The observation that laptop full-mode proving does not complete is — per §4.4 and Tsutsumi [SCIS 2025, §4.1] — a quantitative instantiation of the Gustafson-law scaling limit for zkVM provers on consumer hardware.

6.2 Remaining technical gap

Two to four real-mode prove_ms anchor points are needed to empirically validate the cycles → time extrapolation. Options: rent a cloud VM for ~€20–40 (Hetzner AX162-S or similar); use lab compute if available; or accept extrapolation-only reporting with the theoretical backing from Tsutsumi's Gustafson analysis. **This is a derived resource requirement from prior art, not a project blocker.**

6.3 Specific questions for supervisor review

- **Compute access.** Does the lab have a server available for 2–4 full-mode RISC0 proofs? If not, am I authorised to expense a small cloud rental (~€30) to obtain the anchor points? Or is extrapolation-only reporting with Tsutsumi [SCIS 2025] as theoretical backing acceptable for thesis scope?
- **Scope of B6.** Is the 36-config cycles matrix plus theoretical extrapolation acceptable, or do you require empirical wall-clock anchor points?
- **Scope of B1.** The repaired decoder reports opcode categories and parse timing but not the original B1.3 ACIR → R1CS convert timing (the 1.0 opcode layout differs from 0.x). Is the opcode-category finding sufficient, or should I port the converter?
- **Framing feedback.** §5's step-by-step dissection + the Tsutsumi positioning in §1.5 is the intellectual core. Does this match what you expected from the thesis, or were you expecting a different angle?
- **Timeline.** With ~3 months to submission and the B6 anchor points being the main remaining technical gap, am I on track? What would you like me working on next week?

A written reply at your convenience is preferable — I'd like your direction on these before sinking more time into either cloud compute or further writing.

7. Consolidated data tables

Tables below pull from *results/bench_results.xlsx*, the B6 JSONL (*bench_b6_zkvm_dev_20260416_222710.jsonl*), the B5 / B9 JSONLs in *results/bench_run_20260416_112256/*, and the fresh B1 output (*/tmp/b1_final.jsonl*). Cell colouring is semantic: **red** marks overhead or costs that argue against a backend; **green** marks efficiency or wins; **amber** marks Griffin-specific values (the dominant cost driver across both backends); grey marks neutral / context totals.

7.1 Circuit size — the ~4 000× zkVM expansion

What the prover has to attest to, in the units each backend uses. Aurora counts R1CS constraints; RISC Zero counts rv32im cycles; Noir compiles to ACIR opcodes (a different abstraction layer entirely).

Config	Aurora R1CS constraints	RISC Zero trace cycles	Noir ACIR opcodes	Expansion (zkVM/Aurora)
Loquat verify (hand-written)	246 629	~500 M – 1 B	—	~2 000 – 4 000×
Loquat verify (Noir-compiled)	—	—	**2 866**	(86× vs hand)
BDEC-ShowVer k=1, m=16, rev=20	749 392 [1]	3 789 553 664	—	**~5 000×**
BDEC-ShowVer k=2, m=64, rev=20 (PP2)	996 523	~7 – 8 B (extrap.)	—	**~7 500×**

Table A. [1] Aurora PP3 (no policy) is 749 392 constraints; adding Merkle revocation (PP2) brings the total to 996 523. zkVM per-single-verify is extrapolated from k=1 cycles minus non-verify components.

7.2 Prove / verify / proof-size breakdown

Wall-clock timings across backends and configurations. Aurora numbers are M5 Pro medians of 10 runs (indexer + prove + verify). B8 libiop is a SAT/UNSAT sanity scenario on a padded 2^{18} circuit; it exposes the ZK-variant cost multiplier.

Scenario	Circuit size	Prove (ms)	Verify (ms)	Proof size (B)
AURORA — hand-written R1CS				
single Loquat verify	246 629	**14 077**	**6 260**	8 407 328
PP2 k=2, m=64, rev=20 (no policy)	996 523	**83 015**	51 923	33 573 373
PP3 k=1 + GPA policy	996 814	66 652	37 700	33 573 123
PP3 k=1 + GPA + degree policy	996 821	80 540	49 920	33 572 707
FRACTAL — alternative SNARK				
single Loquat verify	246 629	14 227	6 383	8 409 768
RISC ZERO — zkVM on M-series				
dev-mode k=1, s=1, m=16	3.79 B cyc.	**22 268** [2]	0 [2]	0 [2]
full-mode k=1, s=1, m=16	3.79 B cyc.	> 21 600 000 [3]	—	—
B8 libiop Aurora (padded 262 144)				
λ=128, no ZK	262 144	107 289	—	—
λ=128, with ZK	262 144	**480 093**	—	—
ZK overhead (× over non-ZK)	—	× 4.47	× 1.01	× 1.252

Table B. [2] Dev-mode wall_ms is emulator execute-only (no proof); verify_ms and proof_bytes are 0 by design. [3] Full-mode run on k=1, s=1, m=16 ran for 6+ hours without completing; reported as a lower bound.

7.3 B5 — Griffin dominates Aurora's cost

Per-phase breakdown of Loquat signature-verify constraint cost. Summing Griffin-heavy phases (highlighted amber) gives **99.73%** of the single-verify total. The full credential (Loquat + Merkle + control) is larger, but Griffin still drives ~24.7% of the full-credential cost.

Phase	Δ constraints	% Loquat verify	% full credential	Griffin-heavy?
message commitment	1 595	0.65%	0.16%	**yes**
t-values allocated (witness)	0	—	—	no
quadratic-residuosity	640	0.26%	0.06%	no
transcript binding (Fiat-Shamir)	121 517	**49.29%**	12.19%	**yes**
sumcheck (per-round arith.)	30	<0.01%	<0.01%	no
LDT queries (Merkle auth)	122 752	**49.79%**	12.32%	**yes**
TOTAL (Loquat sig-verify)	246 534	100.00%	24.74%	
TOTAL (full credential)	996 523		100.00%	

Table C. Phase-by-phase constraint deltas from B5 instrumentation. Griffin-heavy = involves Griffin hash calls (message commitment, transcript binding, LDT query Merkle authentication).

7.4 B9 — policy overhead is small (surprising finding)

Adding policy predicates on top of PP3 barely moves the needle. Even the combined GPA + degree policy adds only 298 constraints and < 2% prove time. This is a useful result: **adding policy gating to a post-quantum credential proof does not meaningfully change the cost profile**.

Policy	Constraints	Δ cons.	Prove (ms)	Δ prove	Verify (ms)	Δ verify
none	996 523	—	79 586	—	49 647	—
GPA ≥ 3.0	996 814	+ 291	80 682	**+ 1.38%**	49 762	**+ 0.23%**
GPA + degree ∈ whitelist	996 821	+ 298	80 540	**+ 1.20%**	49 920	**+ 0.55%**

Table D. PP3 policy-gating overhead, M5 Pro, n = 10 runs each.

7.5 Backend interchangeability (Aurora vs Fractal)

Running the same Loquat-verify R1CS through Aurora's alternative SNARK backend (Fractal) produces essentially identical numbers. This rules out "Aurora-specific cost concentration" as an explanation for Griffin dominance — the cost is intrinsic to the FRI-based proof system, not an Aurora quirk.

Metric (single Loquat verify)	Aurora	Fractal	Δ (Fractal vs Aurora)
prove_ms	14 077	14 227	+ 1.1%
verify_ms	6 260	6 383	+ 2.0%
proof_bytes	8 407 328	8 409 768	+ 0.03%

Table E. Aurora vs Fractal on the same hand-written Loquat R1CS, M5 Pro.

7.6 D1 — update churn (the agility argument)

When the protocol changes — new signature scheme, new policy, new attribute schema, new security parameters — how much work does each backend require? This is where the zkVM's flexibility story becomes concrete. Aurora must re-author the R1CS, regenerate ~32 MB of proving / verification keys, and distribute the updated artifacts. RISC Zero rebuilds the guest (seconds to minutes) and distributes a 32-byte ImageID.

Update event	Aurora artifacts	Aurora time	Aurora BW	zkVM artifacts	zkVM time	zkVM BW	BW ratio
Policy update	R1CS + PK + VK	**hours – days**	**33.55 MB**	guest binary	**minutes**	**32 B**	**~ 1 000 000×**
Algorithm rotation	all circuit artifacts	weeks – months	**33.55 MB**	guest + host	**hours – days**	**32 B**	**~ 1 000 000×**
Schema extension	R1CS + keys + Noir src	**days – weeks**	**33.55 MB**	guest binary	**hours**	**32 B**	**~ 1 000 000×**
Parameter refresh	setup params + keys	**hours**	**33.55 MB**	config update	**minutes**	**32 B**	**~ 1 000 000×**

Table F. D1 update-churn analysis — artifact bandwidth is Aurora's verification key + proving key bundle vs RISC Zero's ImageID (a 32-byte hash).

7.7 B1 — Noir compiler overhead

Category breakdown of ACIR opcodes in the nargo-1.0-compiled BDEC-ShowVer. Only 9 of 2 866 opcodes (0.31%) are actual constraint-defining *AssertZero* operations; **99.58% are RANGE checks** enforcing bit-width bounds on intermediate witnesses. This partly explains the 86× gap between Noir-compiled and hand-written R1CS in B2.

ACIR opcode category	Count	% of total	Role
AssertZero	**9**	0.31%	true constraint-defining ops
BlackBoxFuncCall	**2854**	**99.58%**	range / hash / ecc primitives
■■ RANGE (sub-kind)	**2854**		witness bit-width checks
BrilligCall	3	0.10%	unconstrained witness hints
TOTAL	2866	100.00%	

Table G. B1 ACIR opcode breakdown on nargo 1.0.0-beta.19, decoded via the new base64 / gzip / MessagePack decoder at src/noir_backend/acir_binary.rs.

7.8 Cross-backend summary — the headline one-pager

Consolidation of everything above into one comparison. This is the table that will go into the thesis's executive summary.

Dimension	Aurora (hand SNARK)	RISC Zero (zkVM)	Who wins / by how much
Circuit representation	R1CS (hand-authored)	RISC-V execution trace	zkVM: developer ergonomics
Workload size (k=1, m=16)	~1 M constraints	~3.8 B cycles	Aurora: ~3 800× smaller
Prove time (k=1, m=16)	~14 s	> 6 h (did not finish)	Aurora: > 1 500×
Verify time (k=1, m=16)	~6 s	ms-scale (pending)	zkVM: likely winner
Proof size (single Loquat)	~8.4 MB	~200 – 300 kB (STARK seal)	zkVM: ~30× smaller
Proof size (PP2 full cred.)	**~33.6 MB**	—	Aurora: hurts deployment
Verification key size	**~32 MB**	32 B (ImageID)	zkVM: ~1 000 000×
Time to swap signature scheme	weeks – months of R1CS	minutes – days of Rust	zkVM: 100× or more
Update bandwidth (schema change)	**33.6 MB**	32 B (ImageID)	zkVM: ~1 000 000×
Policy overhead (add GPA)	+ 0.03% cons. / + 1.4% prove	native Rust condition	both: minor
Aurora vs Fractal	prove + 1.1%, proof + 0.03%	n / a	either works
Dominant cost driver	Griffin hash (99.73%)	rv32im emulation of crypto	both: hash-bound

Table H. Final Aurora vs RISC Zero comparison across every measured dimension. Green cells mark that backend's win; red cells mark its cost. Amber marks values that are architecture-independent (Griffin dominance).

Report auto-generated from benchmark JSONL records. Data sources: *results/bench_b6_zkvm_dev_20260416_222710.jsonl* (B6), *results/bench_run_20260416_112256/* (B5, B9), */tmp/b1_final.jsonl* (B1), *results/bench_results.xlsx* (cross-validation). Figures and tables produced by *scripts/build_thesis_status_report.py*.